SHRSS – AEM Assets Guide - OpenAPI

Table of Contents

Overview

Using AEM Assets Author API & Folders API for Folder/Asset CRUD

Environment used in examples:

- **Stage author**: https://author-p135156-e1336256.adobeaemcloud.com
- **Base DAM folder**: /content/dam/shrss/corporate/photography

This version of the guide:

- Uses the **OpenAPI-based AEM Assets Author API** and **Folders API** for all examples.
- Assumes **server-to-server OAuth** via **Adobe Developer Console** (ADC).
- Keeps the same CRUD flows as the legacy /api/assets examples:
    - List folder contents
    - Create folder
    - Create/import asset
    - Update asset metadata
    - Delete asset and folder

For background on AEM's OpenAPI-based APIs and server-to-server auth, see:

- AEM APIs overview
- Invoke OpenAPI-based AEM APIs for server-to-server authentication

1 Set up OpenAPI-based AEM Assets APIs

1.1 Prerequisites

You'll need:

- An **IMS Org Admin / System Admin** for SHRSS.
- Access to the **Cloud Manager program** that contains author-p135156-e1336256.
- Access to **Adobe Developer Console** (developer.adobe.com/console).
- Permissions in AEM so the service user can read/write under /content/dam/shrss/corporate/photography.

OpenAPI-based AEM APIs are delivered via the **AEM API gateway** and secured through OAuth S2S as documented in Invoke OpenAPI-based AEM APIs for server to server authentication.

1.2 Create an Adobe Developer Console project

In **Adobe Developer Console** (ADC):

1. **Create a new project** for SHRSS AEM integrations (or reuse an existing one dedicated to AEM APIs).
2. In that project, click **Add API** and add:
    - **AEM Assets Author API**
    - **AEM Folders API** (if exposed as a separate card for your org)
3. When prompted for credentials, choose **OAuth Server-to-Server**.
4. Associate the integration with an appropriate **product profile** that has access to the SHRSS AEM environments.

Record the following from ADC:

- `CLIENT_ID` (a.k.a. API key)
- `CLIENT_SECRET`
- `IMS_ORG` (e.g. `3E1C3D5B5A0C9F1B0A495E0A@AdobeOrg`)
- `TECHNICAL_ACCOUNT_ID` (service principal user)
- SCOPES used for token requests (includes `aem.assets.author`, `aem.folders` as appropriate)

The server-to-server token flow is exactly as described in Invoke OpenAPI-based AEM APIs for server to server authentication.

1.3 Register the Client ID in AEM via Cloud Manager

To allow your ADC client to call the AEM APIs on **author**:

1. In your Git-based configuration repo for Cloud Manager, add an **`api.yaml`** (or update the existing one) under `/conf` (or wherever your program stores API config).
2. Example `api.yaml`:

```yaml
kind: "API"
version: "1"
metadata:
  name: "shrss-aem-openapi-client"
spec:
  apis:
    - name: "aem-assets-author"
      # clientId from ADC (API Key)
      clientId: "<CLIENT_ID>"
```

1. Commit and push this file.
2. Run the **Cloud Manager pipeline** that deploys configuration to AEM (usually the same pipeline that deploys code and OSGi configs).

Once deployed, AEM is aware of your `clientId` and will authorize incoming OpenAPI calls that present:

- An **access token** issued for that client, and
- The matching `X-Api-Key` header with that `clientId`.

For more detail, see "Configure your AEM instance" in Set up OpenAPI-based AEM APIs.

1.4 Verify the service user permissions in AEM

Once the config pipeline is deployed:

1. In **AEM Author (stage)**:
    - Go to **Tools → Security → Users**.
    - Find the user that corresponds to the **technical account** (it usually follows the pattern of the ADC integration name).
2. Ensure that user is in a group with at least:
    - Read/write access to `/content/dam/shrss/corporate/photography`
    - Typical groups: `dam-users`, `dam-administrators`, or a dedicated SHRSS group.

2 Common environment variables & headers (curl)

For the SHRSS **stage author** environment, you can reuse the following shell variables in all examples:

```
export AEM_HOST="https://author-p135156-e1336256.adobeaemcloud.com"

# OAuth 2.0 (S2S) access token – obtained via ADC S2S flow
export ACCESS_TOKEN="<ACCESS_TOKEN>"

# The same clientId you configured in api.yaml
export API_KEY="<CLIENT_ID>"

# Base folder for SHRSS corporate photography
export BASE_FOLDER_PATH="/content/dam/shrss/corporate/photography"
```

Standard headers:

```
-H "Authorization: Bearer $ACCESS_TOKEN"
-H "X-Api-Key: $API_KEY"
-H "Accept: application/json"
```

Token retrieval is not shown in detail here; use the ADC Server-to-Server OAuth flow as documented in Invoke OpenAPI-based AEM APIs for server to server authentication.

3 Folder CRUD with the Folders API

The **Folders API** gives you path-based operations under `/content/dam/...` and is the OpenAPI replacement for the old `/api/assets` folder manipulation.

Reference: AEM Folders API (stable).

3.1 List contents of `/content/dam/shrss/corporate/photography` (Read)

```
curl -s -X GET \
  "$AEM_HOST/adobe/folders/?path=$BASE_FOLDER_PATH&limit=50" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "Accept: application/json"
```

- `path` is the **JCR path** under `/content/dam`.
- Response includes:
  - A `self` section describing the folder itself.
  - A `children` array with both subfolders and assets, depending on the configuration.

You can use this response to traverse folders exactly as you did with `/api/assets/...json`.

3.2 Create a new folder under `photography` (Create – folder)

Target folder:

```
/content/dam/shrss/corporate/photography/api-demo
curl -s -X POST \
  "$AEM_HOST/adobe/folders/" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "path": "/content/dam/shrss/corporate/photography",
    "name": "api-demo",
    "properties": {
      "jcr:title": "API Demo Folder"
    }
  }'
```

Key fields:

- `path`: existing parent folder.
- `name`: name of the new folder.
- `properties`: optional metadata (e.g., `jcr:title`).

Verify:

```
curl -s -X GET \
  "$AEM_HOST/adobe/folders/?path=/content/dam/shrss/corporate/photography/api-demo" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "Accept: application/json"
```

3.3 Delete folder (Delete – folder)

Non-recursive delete (folder must be empty):

```
curl -s -X DELETE \
  "$AEM_HOST/adobe/folders/?path=/content/dam/shrss/corporate/photography/api-demo" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY"
```

If supported by your version of the Folders API, you can pass a **recursive** flag in the body or query string (check the OpenAPI spec). In most cases, you should **first delete assets** within the folder via the Assets API (see below), then remove the folder.

Always confirm with your internal governance whether recursive deletes of DAM content are allowed in stage/prod.

4 Asset CRUD with the Assets Author API

The **Assets Author API** provides OpenAPI operations for assets: list, get metadata, update metadata, delete, and (in some releases) import from URL.

Reference: AEM Assets Author API (stable).

4.1 List assets in `photography` (Read)

You can reuse the **Folders API** for path-based listing (it already returns asset children), or you can use Assets Author API if you need more asset-centric semantics.

Example using Folders API (same as 3.1):

```
curl -s -X GET \
  "$AEM_HOST/adobe/folders/?path=$BASE_FOLDER_PATH&limit=50" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "Accept: application/json"
```

From the `children` array:

- For each asset, capture its `assetId` (or path) which you can use with the Assets Author API.

Depending on your AEM version, you may see fields like `id`, `name`, `path`, `type` (`asset` / `folder`), and `assetId`.

4.2 Create/import an asset (Create – file)

As of today, the **most robust GA mechanism** for binary upload in AEMaaCS remains **Direct Binary Upload** (via the classic API or the `aem-upload` library). The OpenAPI surface is evolving; some tenants have early-access or GA support for **Import from URL**.

If your Assets Author API exposes an **"import from URL"** endpoint (check the OpenAPI spec for something like `POST /adobe/assets/import`), the pattern is:

1. Provide:
    - Target folder path

- Desired asset name
- External URL to fetch
- Optional metadata

2. Poll for completion (if asynchronous).

Example pattern (pseudo-curl – adjust to actual spec):

```
curl -s -X POST \
  "$AEM_HOST/adobe/assets/import" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "targetPath": "/content/dam/shrss/corporate/photography/api-demo",
    "fileName": "sample-openapi.jpg",
    "sourceUrl": "https://example.com/path/to/sample.jpg",
    "properties": {
      "dc:title": "OpenAPI-imported sample",
      "dc:description": "Imported via Assets Author API"
    }
  }'
```

- On success, the response will typically include an `assetId` or a location you can query.

If import-from-URL is not yet available or GA for your environment:

- Use the **classic Direct Binary Upload** (as described in the Classic guide) to place the binary into `/content/dam/shrss/corporate/photography/api-demo/sample.jpg`.
- Then use **Assets Author API** to manage metadata and deletion.

4.3 Get asset metadata (Read – asset)

Assume an asset already exists at:

```
/content/dam/shrss/corporate/photography/api-demo/sample.jpg
```

If your Assets Author API gives you path-based access:

```
curl -s -X GET \
  "$AEM_HOST/adobe/assets?path=/content/dam/shrss/corporate/photography/api-demo/sample.jpg" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "Accept: application/json"
```

If the API is **ID-based**, you'll first resolve path → `assetId` using Folders API listing or a search endpoint, then:

```
curl -s -X GET \
  "$AEM_HOST/adobe/assets/<ASSET_ID>" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
```

```
  -H "X-Api-Key: $API_KEY" \
  -H "Accept: application/json"
```

Inspect the response for `assetMetadata`, `jcr:title`, `dc:title`, and any custom properties.

Exact paths and field names may vary slightly between AEM releases; always cross-check your OpenAPI spec for the `/adobe/assets` endpoints.

4.4 Update asset metadata (Update – asset)

Assume you want to update `dc:title` on `sample.jpg`.

Path-based example (if supported):

```
curl -s -X PATCH \
  "$AEM_HOST/adobe/assets?path=/content/dam/shrss/corporate/photography/api-demo/sample.jpg" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "assetMetadata": {
      "dc:title": "OpenAPI Demo – SHRSS Corporate Photography"
    }
  }'
```

ID-based example:

```
curl -s -X PATCH \
  "$AEM_HOST/adobe/assets/<ASSET_ID>" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "assetMetadata": {
      "dc:title": "OpenAPI Demo – SHRSS Corporate Photography"
    }
  }'
```

Notes:

- You normally supply only the fields you want to update; the API follows a partial-update semantics.
- Check the OpenAPI spec for any constraints around 200 vs 202 responses and job status polling.

4.5 Delete asset (Delete – file)

Path-based delete (if supported):

```
curl -s -X DELETE \
  "$AEM_HOST/adobe/assets?path=/content/dam/shrss/corporate/photography/api-demo/sample.jpg" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY"
```

ID-based delete:

```
curl -s -X DELETE \
  "$AEM_HOST/adobe/assets/<ASSET_ID>" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "X-Api-Key: $API_KEY"
```

After deletion, you can re-list the folder via Folders API (3.1) to confirm the asset is gone.

5 Suggested flow for SHRSS integrations

As the SHRSS senior architect, a practical OpenAPI-based flow for the same CRUD work you previously did with `/api/assets` is:

1. **Authentication**
    - Use ADC **server-to-server OAuth** to obtain `ACCESS_TOKEN` for scopes:
        - `aem.assets.author`
        - `aem.folders`
    - Include:
        - `Authorization: Bearer $ACCESS_TOKEN`
        - `X-Api-Key: $API_KEY` (ADC `clientId`)
2. **Folder navigation**
    - Use **Folders API** to:
        - List children under `/content/dam/shrss/corporate/photography`
        - Create subfolders (e.g. `/api-demo`)
        - Delete subfolders when empty
3. **Asset operations**
    - Prefer **OpenAPI-based import-from-URL** where GA:
        - Provide target path, file name, external URL, and metadata.
    - Where open API upload is not GA, use:
        - **Direct Binary Upload** (classic) for binary placement
        - **Assets Author API** for metadata updates and deletes
4. **Metadata**
    - Use **Assets Author API** (`GET`, `PATCH`) to:
        - Inspect existing metadata (e.g. `dc:title`, `dc:description`)
        - Apply updates in a structured way from your integration
5. **Deletion**
    - Delete the asset with Assets Author API (path- or ID-based).
    - When empty, delete the folder using Folders API.

This keeps the core data model and permissions anchored in AEM, while giving SHRSS a modern, OpenAPI-based surface that is better suited for long-term automation (stronger typing, standard error formats, and discoverable contracts via OpenAPI schema).

6 References

- AEM APIs overview – starting point for all AEM APIs, high-level context on AEM's OpenAPI strategy
- Set up OpenAPI-based AEM APIs – configuring the API gateway, client IDs, and S2S auth
- Invoke OpenAPI-based AEM APIs for server-to-server authentication – full S2S flow and curl examples
- AEM Assets Author API **(OpenAPI)** – detailed reference for asset metadata and operations
- AEM Folders API **(OpenAPI)** – full reference for folder listing, creation, and deletion
- Developer references for Assets – Asset upload **(legacy asset upload & AEM-upload library)** – current preferred approach for direct binary upload until all upload endpoints are GA